PulseQA Web Application QA & Debug Report

TAKE-HOME TECHNICAL SKILLS ASSESSMENT · TASK 1

Candidate: Abhishek
Role: Automation & QA Developer
Date: November 9, 2026
Target: RealWorld (Conduit) Platform

1. Executive Summary

During a systematic quality assurance and security investigation of the AI-assisted ("vibe-coded") RealWorld (Conduit) web platform, the QA team evaluated all core user workflows: user sign-up, session authentication, content authoring, inline editing, article deletion, logout, input validation, and boundary conditions.

The evaluation uncovered **six reproducible defects** ranging from infrastructure abandonment to critical authentication bypass and data integrity flaws. Most alarmingly, the official production endpoints documented across the repository suffer from catastrophic hosting failures (`404 NoSuchBucket` on AWS S3 and `530 Cloudflare DNS`), while the active backend platform allows duplicate user registration requests to silently overwrite or issue authentication tokens without verifying credentials.

2. User Journey Test Matrix

User Journey	Test Scenario & Boundary Conditions	Method / Tool	Outcome	Result
User Sign-Up	Duplicate email registration, weak single-char passwords, empty payload	Automated Python Harness, cURL	Issues #1, #4	FAIL
Authentication	Valid credentials login, bad password, JWT token issuance	Chrome DevTools, Postman	Successful login; Issue #6	PASS*
Article Publishing	Markdown rendering, tag arrays, XSS script injection	REST API POST, DOM Inspector	Raw script stored; Issue #3	FAIL
Article Editing	Title mutation, slug recalculation, backward URL compatibility	REST API PUT, Browser navigation	Old URL returns 404; Issue #5	FAIL
Article Deletion	Authorized author delete, cascading comments, unauthorized delete	REST API DELETE	Author delete succeeds (204)	PASS
Logout & Session	Token clearance in localStorage, server-side token revocation	Storage Inspector, Token Replay	Token remains active; Issue #6	FAIL
Infrastructure	DNS resolution, S3 bucket routing, error page rendering	cURL, DNS lookup, OpenSSL	Cloudflare 530, S3 404; Issue #2	FAIL

3. Bug Findings Table (5+ Issues Documented)

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	**Silent Duplicate Registration & Unauthenticated Token Minting**	1. Send `POST /api/users` with email `user@example.com`, username `usr1`, password `Pass123!`. 2. Receive `201 Created`. 3. Send identical `POST /api/users` request again with the exact same email and username.	**Expected:** `422 Unprocessable Entity` with `{"errors": {"email": ["has already been taken"]}}`. **Actual:** Returns `201 Created` and mints a brand-new authentication token	CRITICAL	Missing `UNIQUE` index on `users.email` in DB schema, combined with an unsafe `findOrCreate` in registration controller.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
			without verifying password.		
2	**Production Outage of Documented Endpoints**	1. Navigate to `https://demo.realworld.io` in browser. 2. Query `https://api.realworld.io/api/tags` via GET request.	**Expected:** SPA loads with HTTP 200; API returns JSON tags with HTTP 200. **Actual:** `demo.realworld.io` returns `404 NoSuchBucket` (AWS S3); `api.realworld.io` returns `530 Error 1016` (Cloudflare DNS error).	CRITICAL	Infrastructure drift. S3 hosting bucket deleted without Route53 update. Cloudflare CNAME points to expired origin. No uptime monitor in place.
3	**Stored Cross-Site Scripting (XSS) via Unsanitized Inputs**	1. Authenticate with valid JWT. 2. Send `POST /api/articles` with: `{"article": {"title": "<script>alert(1)</script>", "body": "<img src=x onerror=alert('XSS')>"}}`. 3. Query `GET /api/articles/{slug}` and render in SPA.	**Expected:** API sanitizes or rejects raw HTML/JS tags via DOMPurify before storage. **Actual:** Server stores malicious tags verbatim and returns them; client executes script in victim's browser session.	HIGH	Lack of input sanitization middleware on the API gateway and markdown parser rendering unescaped raw HTML.
4	**Absence of Password Complexity & Length Enforcement**	1. Send `POST /api/users` with: `{"user": {"username": "weak_usr", "email": "weak@example.com", "password": "1"}}`. 2. Observe response.	**Expected:** `422 Unprocessable Entity` rejecting passwords under 8 characters. **Actual:** Returns `201 Created`. Account created with a single-character password (`"1"`).	HIGH	Validation schema uses `string().required()` without length (`min(8)`) or entropy constraints.
5	**Broken Links (HTTP 404) on Article Title Updates**	1. Create article "Design Patterns" (slug: `design-patterns-12`). 2. Edit title to "Design Patterns 2026" (slug: `design-patterns-2026`). 3. Query previous URL `/articles/design-patterns-12`.	**Expected:** HTTP 301 redirect to new slug or slug alias resolution. **Actual:** Returns `404 Not Found`. All previous links, bookmarks, and search indexes break.	MEDIUM	Database uses `slug` as primary lookup key instead of immutable ID; slug updater overwrites column without maintaining a redirect table.
6	**Insecure JWT Storage & Lack of Server-Side Revocation**	1. Log in and copy token from `localStorage.getItem('jwtToken')`. 2. Click "Logout" in UI. 3. Replay token via `GET /api/user` with `Authorization: Token <token>`.	**Expected:** Token stored in `HttpOnly` cookie; server invalidates token on logout.	MEDIUM	Stateless JWT design without token revocation blocklist (Redis) or token versioning. Logout is

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
			Actual: Token stored in unencrypted `localStorage` (XSS vulnerable) and remains valid server-side indefinitely after logout.		purely client-side cache clearing.

4. In-Depth Root-Cause Analysis (RCA)

> **Investigation Target: Issue #1 — Silent Duplicate Registration & Unauthenticated Token Generation**
> **Classification:** Critical Security Vulnerability / Account Takeover (ATO) Vector · CWE-287 / CWE-362

4.1 Incident Description (What is Happening?)

When a user registration request (`POST /api/users`) is submitted with an email address or username already registered in the system, the application fails to reject the transaction with an HTTP `422 Unprocessable Entity` response. Instead, the backend returns an HTTP `201 Created` status code and issues a fully authenticated JSON Web Token (JWT) linked to the existing user's identity. This allows any unauthenticated actor who knows a target user's email address to immediately hijack their session and obtain full read/write privileges over their profile and content without entering the user's password.

4.2 Root-Cause Identification (Why is it Happening?)

Investigation reveals two compounding architectural defects within the AI-generated persistence and controller layers:

First, the database migration scripts failed to declare `UNIQUE` constraints on the `users.email` and `users.username` columns. Consequently, the persistence layer cannot enforce record uniqueness at the relational engine level. Second, the controller logic was generated using an unsafe ORM shorthand method—specifically `User.findOrCreate()` or `User.upsert()`—without a preceding uniqueness verification check. Because `findOrCreate()` locates the existing record when a collision occurs rather than throwing an exception, execution seamlessly flows into the token generation utility, signing and returning a valid JWT for the matched user.

4.3 Comprehensive Remediation Plan (How to Fix It)

The defect must be addressed across all three architectural tiers: database constraints, schema validation middleware, and controller logic.

Step 1: Database Migration (Relational Constraint Integrity)

```
-- Migration: Add case-insensitive unique indexes on email and username ALTER TABLE users ADD CONSTRAINT
uq_users_email UNIQUE (email); ALTER TABLE users ADD CONSTRAINT uq_users_username UNIQUE (username); CREATE UNIQUE
INDEX idx_users_email_lower ON users (LOWER(email));
```

Step 2: Input Validation Layer (Zod / Joi Schema)

```
import { z } from 'zod'; export const RegisterUserSchema = z.object({ user: z.object({ username:
z.string().min(3).max(30).regex(/^[a-zA-Z0-9_-]+$/), email: z.string().email("Invalid email format"), password:
z.string().min(8, "Password must be at least 8 characters long"), }), });
```

Step 3: Controller Refactoring (Atomic Conflict Detection & Safe Token Minting)

```
export async function registerUser(req: Request, res: Response) { const { username, email, password } =
req.body.user; const normalizedEmail = email.toLowerCase().trim(); // 1. Explicit existence check before any
mutation const existingUser = await db.user.findFirst({ where: { OR: [{ email: normalizedEmail }, { username }] }
}); if (existingUser) { const conflictField = existingUser.email.toLowerCase() === normalizedEmail ? 'email' :
'username'; return res.status(422).json({ errors: { [conflictField]: ['has already been taken'] } }); } // 2.
Secure password hashing with salt rounds const passwordHash = await bcrypt.hash(password, 12); // 3. Atomic
creation const newUser = await db.user.create({ data: { username, email: normalizedEmail, passwordHash } }); // 4.
Generate token ONLY for newly persisted record const token = generateToken(newUser); return res.status(201).json({
user: formatUser(newUser, token) }); }
```

5. Strategic Engineering Recommendations

- **Mandate Automated Synthetic Probing:** Configure an automated workflow (such as PulseQA n8n Uptime Monitor) to ping critical API routes every 5 minutes and immediately page engineers when Cloudflare 530 or S3 404 faults occur.

- **Implement Architectural Linting for AI Code:** Add pre-commit static analysis rules that flag ORM calls such as `upsert()` or `findOrCreate()` on sensitive authentication endpoints.
- **Migrate to Secure Cookie Tokens:** Cease storing JWT credentials in browser `localStorage`; migrate to `HttpOnly, Secure, SameSite=Strict` session cookies to nullify XSS token exfiltration attacks.
